

1 Most Interesting Design / UX Decision

The House X-Ray live cost map. Instead of a flat checklist with a total at the bottom, the estimate renders as a labeled cross-section of a house (roof, a systems band, room boxes, a foundation strip) that heats up orange in real time as cost accrues to each zone, so the biggest driver is visible at a glance rather than buried in a list. Tapping a zone jumps to that room and lights it up with a soft glow, while the room chips and the item list below stay in sync. It turns an abstract dollar figure into something an agent can read like a heat map *while standing in the property*. What makes it cheap is the flat data model underneath: the whole estimate is one cells map keyed by "roomId|groupId|itemId", so each zone total is a filter + reduce over matching keys, recomputed on every render with no tree traversal and no extra bookkeeping. That same flat model makes the single render() pass, plus snapshots and version diffing, nearly free (a snapshot is one deep copy; comparing versions is a set-difference over keys).

2 What Is Broken or Fragile

Two honest trade-offs I made for the offline, single-file constraint, each with a clear path to harden:

- **localStorage as the datastore.** Photos are base64-encoded inline, so a heavy inspection with 50+ high-res photos can approach the ~5 MB localStorage limit on some browsers. I compress images to ≤ 1280 px and guard every write with a quota check, so it fails safe rather than silently, but the real fix is moving photo blobs to IndexedDB. That is the first thing I would harden for production.
- **Full re-render on every mutation.** Rebuilding the whole screen from state keeps the code simple and predictable, but it can cost scroll position and input focus on rapid interactions. I already added targeted partial re-renders for the hot paths (room switching, deal inputs, the contingency slider); a keyed reconciliation step would remove the remaining jank in the general case.

3 Creative Addition & Why

The Deal Analyzer + AI Copilot layer, built as a single creative system, not a cosmetic feature.

- **Deal Analyzer:** The brief says the repair estimate is the most important number, but an estimate in isolation doesn't answer the real question: "*Should I buy this house?*" The Deal Analyzer pulls the live repair total, adds ARV, purchase price, holding costs, and selling costs, then projects profit, ROI, and a 70%-rule max offer with a GO / CAUTION / NO-GO verdict. The agent gets a buy-or-walk decision *while still standing in the house*.
- **AI Copilot:** Four specialist modules: Inspection Coverage (flags missed big-ticket systems), Recommendations (commonly-paired work like "flooring → trim-out"), Anomaly & Risk (outlier quantities, price-override fraud, conflicting scope), and a semantic chat Assistant backed by on-device embeddings (MiniLM) over a repair-guidelines knowledge base. All 100% offline, no API key, no server. I chose this because the brief's multi-agent/RAG requirements shouldn't be faked with canned responses; this delivers the architecture and value on-device.
- **Voice Walkthrough + Camera Scope Detection:** Narrate repairs hands-free ("kitchen, cabinets, five doors"); on-device Whisper transcribes, a synonym+semantic matcher adds items with correct quantities. Point the camera at a fixture; CLIP classifies it and suggests the line item. Both work offline.
- **Field-trust layer:** because an agent is betting real money, I added a *contingency buffer* (slider or typed %, folded into the profit/verdict so a "GO" can't hide a thin margin), *estimate history with named snapshots* (compare any snapshot to now, item by item, to prove the scope wasn't inflated after a contractor re-walk), and timestamped, room-tagged *inspection notes* carried into the report and Excel. These turn a calculator into a tool an agent would actually trust.
- **Backendless handoff:** a finished estimate compresses into the URL hash and shows as a *QR code or copy-link*, so it imports onto another agent's phone with no server; a one-tap *email hand-off* drops a formatted summary (totals, section breakdown, verdict, notes) to the office. Sharing and reporting without any backend keeps the whole product inside the offline rule.

4 What I'd Ship Next (Two More Days)

- **AR room capture (WebXR).** Use the phone camera and WebXR depth to measure a room live, auto-filling square-footage quantities (flooring, paint, tile) instead of hand-entering them, and let the agent pin repair annotations in 3D space that replay as a walkthrough the office can review. It extends the on-device-vision theme from "classify a fixture" to "measure and map the whole room," and would still run without a server. The two-day version targets one AR measurement flow feeding the flooring and paint line items.
- **Comp-based ARV assistant.** Integrate a public comps source (Redfin/Zillow API or a scraped feed) so the Deal Analyzer auto-suggests an ARV from recent sold comps instead of relying on manual input, shown on a map with price/sqft overlays. This closes the last manual gap in the deal math flow. (Alongside this, the production hardening from section 2: photo blobs in IndexedDB, then optional multi-device sync.)



Role of AI Tools in Development

AI coding assistants (Claude, Gemini) acted as a **pair-programming partner** throughout: architecture brainstorming (the flat-key data model, the layered Copilot resolver, the offline-first service worker), prototyping boilerplate (the 108-item price map, the SVG charts, the semantic-search pipeline), debugging (re-render scroll bugs, iOS PWA quirks, Transformers.js WASM issues), and drafting the README, ARCHITECTURE, and this writeup. All AI-generated code was reviewed, tested, and iterated by hand: the AI accelerated implementation of decisions I made, it did not autonomously design features or make product calls.